MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Gendal Lal

Subject & Subject Code: Theory of Computation & 4CSE4-03, 4AM4-03, 4AD4-03

Branch: CSE, AIML and AI & DS

Batch:

Semester: IV

Session: 2023-24(Even Sem.)

Unit No.: 2

Date of Submission: 22/02/2024

Lect.No.	Topics Covered	Page No.	Total pages
1	Context free grammars(CFG)	1-7	7
2	Ambiguity in CFG	8-9	2
3	Closure of CFG	10-11	2
4	CFG Simplification	12-15	4
5	CNF	16-18	3
6	Chomsky Classification of Grammars	19-21	3
7	Greibach Normal form	22-23	5
8	GNF including membership problem	24-24	1

References Book:

1. "Introduction to the Theory of Computation "by Micheal Sipser
2. "Introduction to computability "by Fred C Henie

G.Lal
FACULTY

HOD

22/2/2024
DEAN ACADEMICS

UNIT-2 (TOC)

LECTURE NO:1

Context-Free Grammar

Definition – A context-free grammar (CFG) consisting of a finite set of grammar rules is a quadruple **(N, T, P, S)** where,

- **N** is a set of non-terminal symbols.
- **T** is a set of terminals where **N ∩ T = NULL.**
- **P** is a set of rules, **P: N → (N ∪ T)***, i.e., the left-hand side of the production rule P does have any right context or left context.
- **S** is the start symbol.

Example

- The grammar ({A}, {a, b, c}, P, A), P : A → aA, A → abc.
- The grammar ({S, a, b}, {a, b}, P, S), P: S → aSa, S → bSb, S → ε
- The grammar ({S, F}, {0, 1}, P, S), P: S → 00S | 11F, F → 00F | ε

Generation of Derivation Tree

A derivation tree or parse tree is an ordered rooted tree that graphically represents the semantic information a string derived from a context-free grammar.

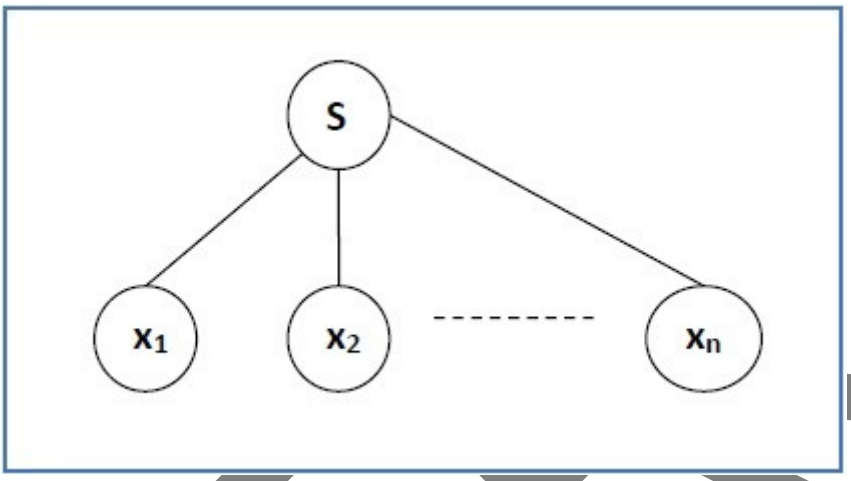
Representation Technique

MITRC

- **Root vertex** − Must be labeled by the start symbol.

- **Vertex** − Labeled by a non-terminal symbol.
- **Leaves** − Labeled by a terminal symbol or ε.

If S → x1x2 …… xn is a production rule in a CFG, then the parse tree / derivation tree will be as follows −



Derivation Tree

There are two different approaches to draw a derivation tree −

1. Top-down Approach −

- Starts with the starting symbol S
- Goes down to tree leaves using productions

2. Bottom-up Approach −

- Starts from tree leaves
- Proceeds upward to the root which is the starting symbol S

MITS

Derivation or Yield of a Tree

The derivation or the yield of a parse tree is the final string obtained by concatenating the labels of the leaves of the tree from left to right, ignoring the Nulls. However, if all the leaves are Null, derivation is Null.

Example

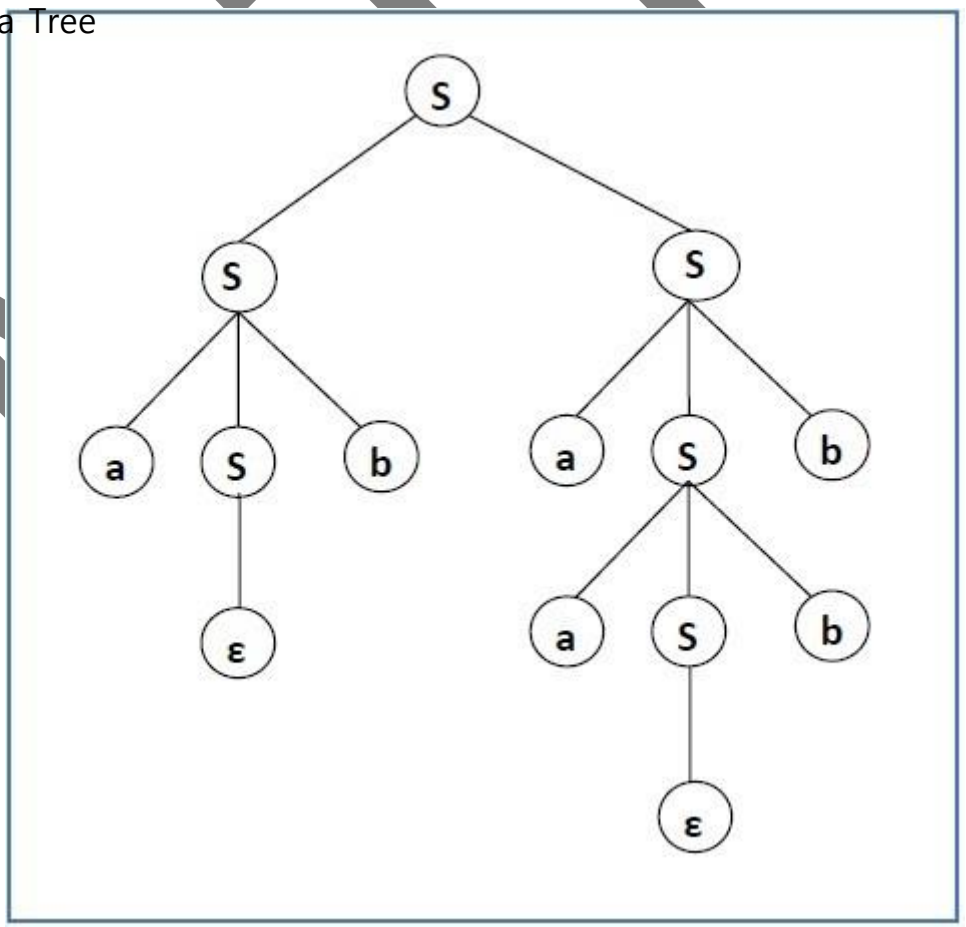
Let a CFG {N,T,P,S} be

N = {S}, T = {a, b}, Starting symbol = S, P = S → SS | aSb | ε

One derivation from the above CFG is "abaabb"

S → SS → aSbS → abS → abaSb → abaaSbb → abaabb

Yield of a Tree



Sentential Form and Partial Derivation Tree

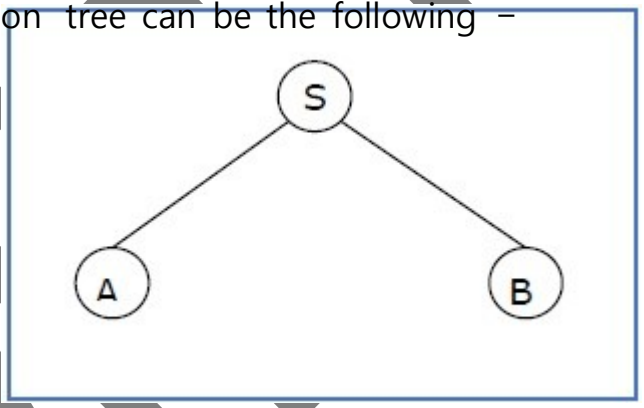
A partial derivation tree is a sub-tree of a derivation tree/parse tree such that either all of its children are in the sub-tree or none of them are in the sub-tree.

Example

If in any CFG the productions are − S

→ AB, A → aaA | ε, B → Bb| ε

the partial derivation tree can be the following −



Sentential Form and Partial Derivation Tree

If a partial derivation tree contains the root S, it is called a sentential form. The above sub-tree is also in sentential form.

Leftmost and Rightmost Derivation of a String

- **Leftmost derivation** − A leftmost derivation is obtained by applying production to the leftmost variable in each step.
- **Rightmost derivation** − A rightmost derivation is obtained by applying production to the rightmost variable in each step.

Example

Let any set of production rules in a CFG be

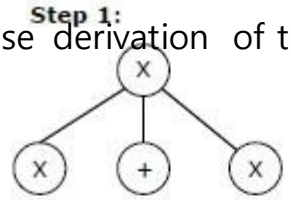
$$X \rightarrow X+X \mid X*X \mid X \mid a$$

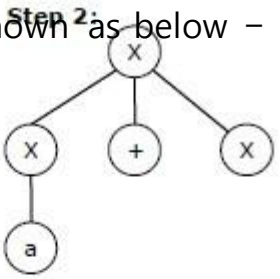
over an alphabet {a}.

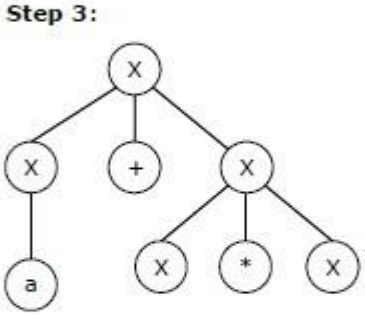
The leftmost derivation for the string "a+a*a" may be − X

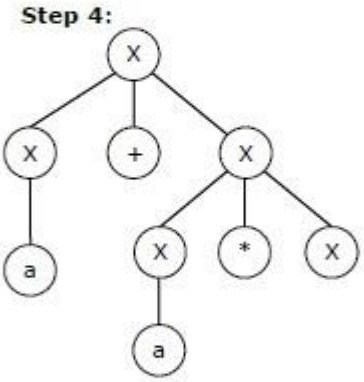
$$\rightarrow X+X \rightarrow a+X \rightarrow a + X*X \rightarrow a+a*X \rightarrow a+a*a$$

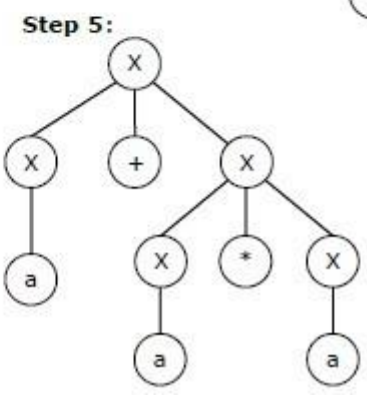
The stepwise derivation of the above string is shown as below −











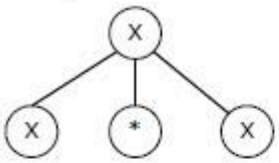
Notes By: i f,

The rightmost derivation for the above string "a+a*a" may be

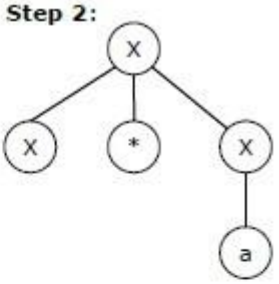
X → X*X → X*a → X+X*a → X+a*a → a+a*a

The stepwise derivation of the above string is shown as below −

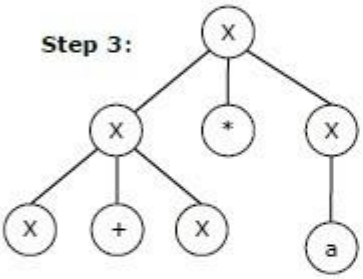
Step 1:



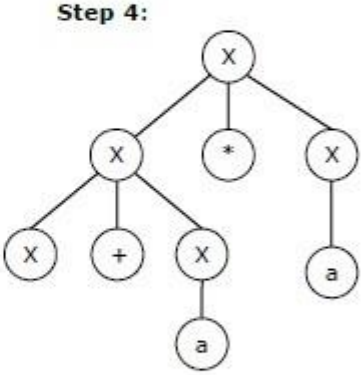
Step 2:



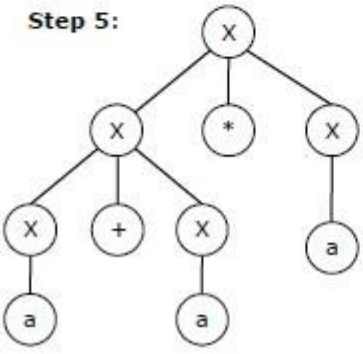
Step 3:



Step 4:



Step 5:



Left and Right Recursive Grammars

In a context-free grammar G, if there is a production in the form X → Xa where X is a non-terminal and 'a' is a string of terminals, it is called a left recursive production. The grammar having a left recursive production is called a left recursive grammar.

And if in a context-free grammar G, if there is a production is in the form X → aX where X is a non-terminal and 'a' is a string of terminals, it is called a right recursive production. The grammar having a right recursive production is called a right recursive grammar.

MITRC

LECTURE NO:-2

Ambiguity in Context-Free Grammars

If a context free grammar G has more than one derivation tree for some string w ∈ L(G), it is called an ambiguous grammar. There exist multiple right-most or left-most derivations for some string generated from that grammar.

Problem

Check whether the grammar G with production rules − X

→ X+X | X*X |X| a

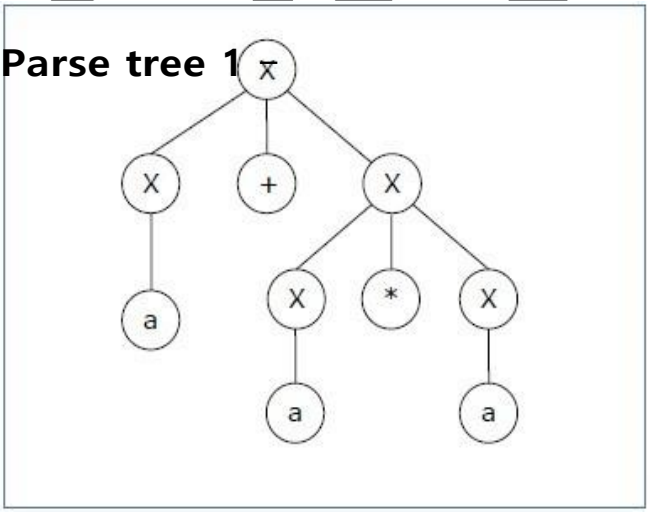
is ambiguous or not.

Solution

Let's find out the derivation tree for the string "a+a*a". It has two leftmost derivations.

Derivation 1 − X → X+X → a +X → a+ X*X → a+a*X → a+a*a

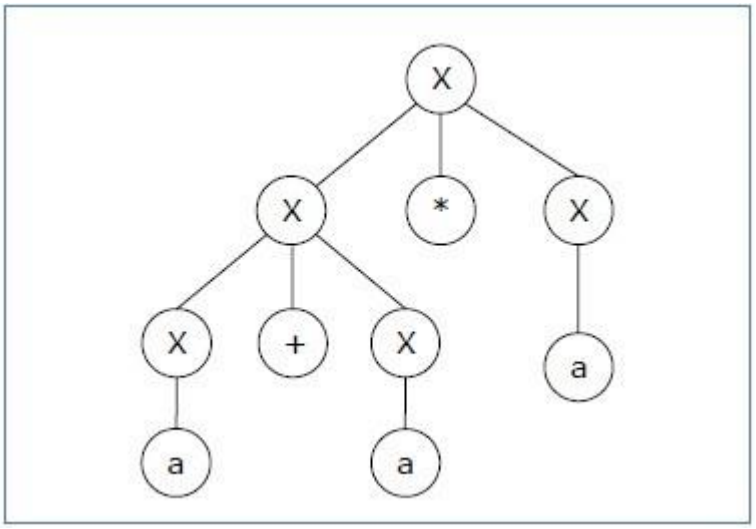
Parse tree 1



NITRC

Derivation 2 – X → X*X → X+X*X → a+ X*X → a+a*X → a+a*a

Parse tree 2 –



Since there are two parse trees for a single string "a+a*a", the grammar **G** is ambiguous.

MITRC

LECTURE NO:-3

CFL Closure Property

Context-free languages are **closed** under −

- Union
- Concatenation
- Kleene Star operation

Union

Let L1 and L2 be two context free languages. Then L1 ∪ L2 is also context free.

Example

Let L1 = { anbn , n > 0}. Corresponding grammar G1 will have P: S1 → aAb|ab

Let L2 = { cmdm , m ≥ 0}. Corresponding grammar G2 will have P: S2 → cBb| ε

Union of L1 and L2, L = L1 ∪ L2 = { anbn } ∪ { cmdm }

The corresponding grammar G will have the additional production S → S1 | S2

Concatenation

If L1 and L2 are context free languages, then L1L2 is also context free.

Example

Union of the languages L1 and L2, L = L1L2 = { anbncmdm }

The corresponding grammar G will have the additional

MITRC

production S → S1 S2

Kleene Star

If L is a context free language, then L* is also context free.

Example

Let L = { anbn , n ≥ 0}. Corresponding grammar G will have P: S → aAb| ε

Kleene Star L1 = { anbn }*

The corresponding grammar G1 will have additional productions

S1 → SS1 | ε

Context-free languages are not closed under −

- Intersection − If L1 and L2 are context free languages, then L1 ∩ L2 is not necessarily context free.
- Intersection with Regular Language − If L1 is a regular language and L2 is a context free language, then L1 ∩ L2 is a context free language.
- Complement − If L1 is a context free language, then L1′ may not be context free.

MTRC

LECTURE NO:-4

CFG Simplification

In a CFG, it may happen that all the production rules and symbols are not needed for the derivation of strings. Besides, there may be some null productions and unit productions. Elimination of these productions and symbols is called **simplification of CFGs**. Simplification essentially comprises of the following steps −

- Reduction of CFG
- Removal of Unit Productions
- Removal of Null Productions

Reduction of CFG

CFGs are reduced in two phases −

Phase 1 − Derivation of an equivalent grammar, **G'**, from the CFG, **G**, such that each variable derives some terminal string.

Derivation Procedure −

Step 1 − Include all symbols, **W1**, that derive some terminal and initialize **i = 1**.

Step 2 − Include all symbols, **Wi+1**, that derive **Wi**.

Step 3 − Increment **i** and repeat Step 2, until **Wi+1 = Wi**.

Step 4 − Include all production rules that have **Wi** in it.

Phase 2 − Derivation of an equivalent grammar, **G"**, from the CFG, **G'**, such that each symbol appears in a sentential form.

Derivation Procedure −

Step 1 − Include the start symbol in **Y1** and initialize **i = 1**.

MITRC

Step 2 − Include all symbols, **Yi+1**, that can be derived from **Yi** and include all production rules that have been applied.

Step 3 − Increment **i** and repeat Step 2, until **Yi+1 = Yi**.

Problem

Find a reduced grammar equivalent to the grammar G, having production rules, P: S → AC | B, A → a, C → c | BC, E → aA | e

Solution

Phase 1 −

T = { a, c, e }

W1 = { A, C, E } from rules A → a, C → c and E → aA W2

= { A, C, E } U { S } from rule S → AC

W3 = { A, C, E, S } U ∅

Since W2 = W3, we can derive G′ as − G′ =

{ { A, C, E, S }, { a, c, e }, P, {S}} where P: S →

AC, A → a, C → c , E → aA | e **Phase 2** −

Y1 = { S }

Y2 = { S, A, C } from rule S → AC

Y3 = { S, A, C, a, c } from rules A → a and C → c

Y4 = { S, A, C, a, c }

Since Y3 = Y4, we can derive G″ as − G″

= { { A, C, S }, { a, c }, P, {S}} where P: S

→ AC, A → a, C → c

Removal of Unit Productions

Any production rule in the form A → B where A, B ∈ Non-terminal is called **unit production.**.

MITRC

Removal Procedure −

Step 1 − To remove **A → B**, add production **A → x** to the grammar rule whenever **B → x** occurs in the grammar. [x ∈ Terminal, x can be Null]

Step 2 − Delete **A → B** from the grammar.

Step 3 − Repeat from step 1 until all unit productions are removed.

Problem

Remove unit production from the following − S → XY, X → a, Y → Z | b, Z → M, M → N, N → a

Solution −

There are 3 unit productions in the grammar − Y → Z, Z → M, and M → N

At first, we will remove M → N.

As N → a, we add M → a, and M → N is removed.

The production set becomes

S → XY, X → a, Y → Z | b, Z → M, M → a, N → a

Now we will remove Z → M.

As M → a, we add Z→ a, and Z → M is removed.

The production set becomes

S → XY, X → a, Y → Z | b, Z → a, M → a, N → a

Now we will remove Y → Z.

As Z → a, we add Y→ a, and Y → Z is removed.

The production set becomes

MITRC

S → XY, X → a, Y → a | b, Z → a, M → a, N → a

Now Z, M, and N are unreachable, hence we can remove those.

The final CFG is unit production free −

S → XY, X → a, Y → a | b

Removal of Null Productions

In a CFG, a non-terminal symbol '**A'** is a nullable variable if there is a production **A → ε** or there is a derivation that starts at **A** and finally ends up with

ε: A → ..... → ε

Removal Procedure

Step 1 − Find out nullable non-terminal variables which derive ε.

Step 2 − For each production **A → a**, construct all productions **A → x** where **x** is obtained from '**a'** by removing one or multiple non-terminals from Step 1.

Step 3 − Combine the original productions with the result of step 2 and remove **ε - productions**.

Problem

Remove null production from the following − S

→ ASA | aB | b, A → B, B → b | ∈

Solution

–

There are two nullable variables − **A** and **B**

At first, we will remove B → ε.

After removing **B → ε**, the production set becomes −

S→ASA | aB | b | a, A ε B| b | &epsilon, B → b

Now we will remove A → ε.

After removing **A → ε**, the production set becomes −

S→ASA | aB | b | a | SA | AS | S, A → B| b, B → b

MITRC

This is the final production set without null transition.

LECTURE NO:-5

Chomsky Normal Form

A CFG is in Chomsky Normal Form if the Productions are in the following forms −

- A → a
- A → BC
- S → ε

where A, B, and C are non-terminals and **a** is terminal.

Algorithm to Convert into Chomsky Normal Form −

Step 1 − If the start symbol **S** occurs on some right side, create a new start symbol **S'** and a new production **S'→ S**.

Step 2 − Remove Null productions. (Using the Null production removal algorithm discussed earlier)

Step 3 − Remove unit productions. (Using the Unit production removal algorithm discussed earlier)

Step 4 − Replace each production **A → B1…Bn** where **n > 2** with **A → B1C** where **C → B2 …Bn**. Repeat this step for all productions having two or more symbols in the right side.

Step 5 − If the right side of any production is in the form **A → aB** where a is a terminal and **A, B** are non-terminal, then the production is replaced by **A → XB** and **X → a**. Repeat this step for every production which is in the form **A → aB**.

Problem

Convert the following CFG into CNF S

→ ASA | aB, A → B | S, B → b | ε

Solution

(1) Since **S** appears in R.H.S, we add a new state **S0** and **S0→S** is added to the production set and it becomes −

MTRC

S0→S, S→ ASA | aB, A → B | S, B → b | ∈

(2) Now we will remove the null productions −

B → ∈ and A → ∈

After removing B → ε, the production set becomes −

S0→S, S→ ASA | aB | a, A → B | S | ∈, B → b

After removing A → ∈, the production set becomes −

S0→S, S→ ASA | aB | a | AS | SA | S, A → B | S, B → b **(3)**

Now we will remove the unit productions.

After removing S → S, the production set becomes −

S0→S, S→ ASA | aB | a | AS | SA, A → B | S, B → b After

removing S0→ S, the production set becomes − S0→

ASA | aB | a | AS | SA, S→ ASA | aB | a | AS | SA

A → B | S, B → b

After removing A→ B, the production set becomes − S0

→ ASA | aB | a | AS | SA, S→ ASA | aB | a | AS | SA A → S | b

B → b

After removing A→ S, the production set becomes − S0

→ ASA | aB | a | AS | SA, S→ ASA | aB | a | AS | SA A → b

|ASA | aB | a | AS | SA, B → b

(4) Now we will find out more than two variables in the R.H.S Here,

S0→ ASA, S → ASA, A→ ASA violates two Non-terminals in

R.H.S.

Hence we will apply step 4 and step 5 to get the following final production set which is in CNF −

S0→ AX | aB | a | AS | SA

MITRC

S→ AX | aB | a | AS | SA

A → b |AX | aB | a | AS | SA

B → b

X → SA

(5) We have to change the productions S0→ aB, S→ aB, A→ aB

And the final production set becomes −

S0→ AX | YB | a | AS | SA S→

AX | YB | a | AS | SA

A → b A → b |AX | YB | a | AS | SA B →

b

X → SA

Y → a

MITRC

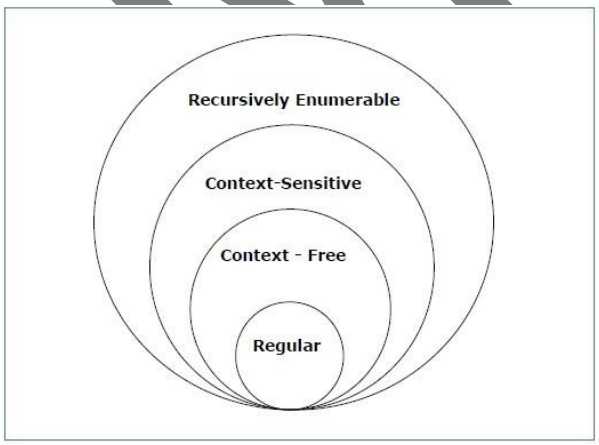
LECTURE -6

Chomsky Classification of Grammars

According to Noam Chomosky, there are four types of grammars − Type 0, Type 1, Type 2, and Type 3. The following table shows how they differ from each other −

Grammar Type	Grammar Accepted	Language Accepted	Automaton
Type 0	Unrestricted grammar	Recursively enumerable language	Turing Machine
Type 1	Context-sensitive grammar	Context-sensitive language	Linear-bounded automaton
Type 2	Context-free grammar	Context-free language	Pushdown automaton
Type 3	Regular grammar	Regular language	Finite state automaton

Take a look at the following illustration. It shows the scope of each type of grammar −



Containment of Type3, Type2, Type1, Type0

MITRC

Type - 3 Grammar

Type-3 grammars generate regular languages. Type-3 grammars must have a single non-terminal on the left-hand side and a right-hand side consisting of a single terminal or single terminal followed by a single non-terminal.

The productions must be in the form $X \rightarrow a$ or $X \rightarrow aY$

where $X, Y \in N$ (Non terminal)

and $a \in T$ (Terminal)

The rule $S \rightarrow \varepsilon$ is allowed if S does not appear on the right side of any rule.

Example

$X \rightarrow \varepsilon$

$X \rightarrow a \mid aY$

$Y \rightarrow b$

Type - 2 Grammar

Type-2 grammars generate context-free languages. The productions must be in the form $A \rightarrow \gamma$

where $A \in N$ (Non terminal)

and $\gamma \in (T \cup N)^*$ (String of terminals and non-terminals).

These languages generated by these grammars are be recognized by a non-deterministic pushdown automaton.

Example

$S \rightarrow X\ a$

$X \rightarrow a$

$X \rightarrow aX\ X$

MITRC

→ abc X

→ ε

Type - 1 Grammar

Type-1 grammars generate context-sensitive languages. The productions must be in the form

$\alpha A \beta \rightarrow \alpha \gamma \beta$

where $A \in N$ (Non-terminal)

and $\alpha, \beta, \gamma \in (T \cup N)^*$ (Strings of terminals and non-terminals)

The strings α and β may be empty, but γ must be non-empty.The rule $S \rightarrow \varepsilon$ is allowed if S does not appear on the right side of any rule. The languages generated by these grammars are recognized by a linear bounded automaton.

Example

$AB \rightarrow AbBc$

$A \rightarrow bcA$

$B \rightarrow b$

Type - 0 Grammar

Type-0 grammars generate recursively enumerable languages. The productions have no restrictions. They are any phase structure grammar including all formal grammars.

They generate the languages that are recognized by a Turing machine.

The productions can be in the form of $\alpha \rightarrow \beta$ where α is a string of terminals and nonterminals with at least one non-terminal and α cannot be null. β is a string of terminals and non-terminals.

Example

$S \rightarrow ACaB$

$Bc \rightarrow acB$

MITRC

CB → DB

aD → Db

LECTURE-7

Greibach Normal Form

A CFG is in Greibach Normal Form if the Productions are in the following forms −

A → b

A → bD1…Dn

S → ε

where A, D1,....,Dn are non-terminals and b is a terminal.

Algorithm to Convert a CFG into Greibach Normal Form

Step 1 − If the start symbol **S** occurs on some right side, create a new start symbol **S'** and a new production **S' → S**.

Step 2 − Remove Null productions. (Using the Null production removal algorithm discussed earlier)

Step 3 − Remove unit productions. (Using the Unit production removal algorithm discussed earlier)

Step 4 − Remove all direct and indirect left-recursion.

Step 5 − Do proper substitutions of productions to convert it into the proper form of GNF.

Problem

Convert the following CFG into CNF S

→ XY | Xn | p

X → mX | m

Y → Xn | o

MITRC

Solution

Here, **S** does not appear on the right side of any production and there are no unit or null productions in the production rule set. So, we can skip Step 1 to Step 3.

Step 4

Now after replacing

X in S → XY | Xo | p

with

mX | m

we obtain

S → mXY | mY | mXo | mo | p.

And after replacing

X in Y → Xn | o

with the right side of

X → mX | m

we obtain

Y → mXn | mn | o.

Two new productions O → o and P → p are added to the production set and then we came to the final GNF as the following −

S → mXY | mY | mXC | mC | p

X → mX | m

Y → mXD | mD | o

O → o

P → p

MITRC

LECTURE-8

Pumping Lemma for CFG

Lemma

If **L** is a context-free language, there is a pumping length **p** such that any string **w** ∈ **L** of length ≥ **p** can be written as **w = uvxyz**, where **vy ≠ ε**, |**xy**| ≤ **p**, and for all **i ≥ 0, uvixyiz ∈ L**.

Applications of Pumping Lemma

Pumping lemma is used to check whether a grammar is context free or not. Let us take an example and show how it is checked.

Problem

Find out whether the language **L = {xnynzn | n ≥ 1}** is context free or not.

Solution

Let **L** is context free. Then, **L** must satisfy pumping lemma.

At first, choose a number **n** of the pumping lemma. Then, take z as 0n1n2n.

Break **z** into **uvwxy,** where

|wx| ≤ n and vx ≠ ε.

Hence **vwx** cannot involve both 0s and 2s, since the last 0 and the first 2 are at least (n+1) positions apart. There are two cases –

Case 1 − **vwx** has no 2s. Then **vx** has only 0s and 1s. Then **uwy**, which would have to be in **L**, has **n** 2s, but fewer than **n** 0s or 1s.

Case 2 − **vwx** has no 0s.

Here contradiction occurs.

MITRC

Hence, **L** is not a context-free language.

MITRC